# Chapter 3: Implementation and Results to Date

LeakLens, Software Engineering Minor Project, Semester V. Progress report of 12 November 2026.

| | |
|---|---|
| **Project** | LeakLens: Static and Dynamic Detection of Data Leakage in Machine Learning Code |
| **Theme** | Software Engineering (Minor Project, Semester V) |
| **Submitted by** | Srijeet Prasad Banerjee, B.Tech IT, 3rd Year |
| **Supervisor** | Prof. Tridib Chakraborty |
| **Institute** | Guru Nanak Institute of Technology, Kolkata |
| **Repository** | https://github.com/bytes06runner/LeakLens (all results below are logged in results/) |

## 3.1 Summary

Chapters 1 and 2 proposed a tool that finds data leakage in Python ML code, confirms it on the data, and measures how much each leak inflates the reported score. On 29 September the repository held the taxonomy, the specification, a sandbox and a labelled benchmark, and **no detector**. This chapter reports what exists on 12 November: a working static analyser for all six leakage types, a dynamic (instrumented-run) pass, per-finding impact estimation, a command-line tool with JSON and SARIF output, and first results.

**The main result so far, on the development (DEV) half of the injected benchmark:** reading the code alone, LeakLens finds 80 of 89 labelled leaks with 0 false positives; with the instrumented run added it finds 89 of 89 with 0 false positives, and raises no finding on 10 of 10 files that have no leak. **These are DEV numbers.** The rules were written while looking at these files, so they show that the method works end to end; they are not an estimate of accuracy on unseen code. The held-out TEST half has not been scored, no real notebook has been labelled, and no other tool has been run on our benchmark yet. Section 3.7 lists what went wrong or is still missing.

## 3.2 Progress against the plan

| Phase | Deliverable | 29 September | 12 November |
|---|---|---|---|
| 0 | Setup, novelty check, baseline runnability | done | done; novelty search repeated today (3.7) |
| 1 | Taxonomy of six types, specification, sandbox | done | done |
| 2 | Benchmark: 175 injected files (162 leaks) and 150 real notebooks, DEV/TEST split frozen before any rule | injected done; real set sampled, unlabelled | unchanged: real notebooks still need two human labellers |
| 3 | Evaluation protocol | not started | written as a proposal; **not frozen**, waits for approval |
| 4 | Static analyser | not started | **built**: all six types, 150+ rule tests |
| 5 | Dynamic analyser | not started | **built**: confirms, refutes, and finds data-only leaks |
| 6 | Impact estimation | not started | **built** for six kinds of fix; about half of the leaks have no automatic fix yet |
| 7 | Evaluation on TEST, baselines, case study | not started | DEV only; TEST untouched; CalibraEEG case study run (static): negative result |
| 8 | Mining study of public notebooks | not started | not started |
| 9 | Release (PyPI, pre-commit, GitHub Action) | not started | CLI, JSON, SARIF done; packaging not started; name clash found (3.7) |

## 3.3 What was built

| Component | What it does |
|---|---|
| Ingest | Reads .py and .ipynb files into one module with a map back to cell and line. Uses execution order when every cell has one, else cell order, and records which. Files that do not parse are reported, never silently skipped. |
| Static analyser | One pass of abstract interpretation over pandas, NumPy, scikit-learn and imbalanced-learn code. For every variable it tracks which table the rows came from, which side of which split they are on, which columns they hold and what has leaked into them. Rules fire where rows are split, a model is fitted and scored, or a best score is selected. The analysed code is never run. |
| Dynamic pass | Runs the script with scikit-learn instrumented. At every split it measures, fold by fold, identical rows on both sides, entity values on both sides, and training rows dated after evaluation rows. Entity and time columns are found in the script's own variables, so **the user declares nothing**. |
| Impact estimation | Re-runs the user's own script with one call site changed (group-aware or chronological split, evaluation rows removed from a fit, a transformer fitted on training rows only, duplicates or one feature removed) and reports the change in the metric the script itself prints. |
| Reporting | Terminal output, JSON (validated against a schema), SARIF 2.1.0 for GitHub code scanning, CI exit codes, inline suppression comments. |
| Evaluation | One matching rule and bootstrap confidence intervals shared by every tool; a runner that refuses to touch TEST until the protocol is frozen. |

*About 5,200 lines of Python in the package and 1,500 lines of tests (359 tests pass, including property-based tests that the analyser never crashes and is deterministic).*

## 3.4 A worked example

A DEV pipeline loads a table of patients from 12 hospital sites, keeps four marker columns and calls `train_test_split`. The code never mentions the column `site`, so no static tool can see a problem. LeakLens output for that file (static + dynamic + impact):

```
line 17 [high/high] group (LL-GRP-004) run: confirmed test_auc 0.6865 -> 0.3016 with the fix
The data has a column `site` that the split ignores: 12 of its values are on both sides, and 100% of the evaluation rows belong to an entity that is also in the training rows.
fix: Split by entity: use GroupShuffleSplit with groups=`site` (same folds, test size and seed).
```

*This is the sentence Chapter 1 asked for ("line 42 lets the same patient into both training and test, and fixing it drops your score from X to Y"). The size of the drop here is a property of our synthetic data, not evidence about real studies; and section 3.6 shows this particular kind of estimate is the least accurate one.*

## 3.5 Detection results on the DEV split

Injected benchmark, DEV half: 96 files generated from 7 base pipelines, 89 labelled leaks, 10 files without a leak (7 clean, 3 traps that look leaky but are correct). A finding counts only if it has the right type and overlaps the labelled lines in the right cell. Findings of medium or high confidence are counted. Intervals: bootstrap over files, 2,000 resamples.

| Leakage type | Labelled leaks | Static: found | Static: recall (95% CI) | Static + dynamic: found | Static + dynamic: recall (95% CI) | False positives (either mode) |
|---|---|---|---|---|---|---|
| Preprocessing | 30 | 30 | 1.00 (1.00 to 1.00) | 30 | 1.00 (1.00 to 1.00) | 0 |
| Overlap / duplicates | 10 | 8 | 0.80 (0.50 to 1.00) | 10 | 1.00 (1.00 to 1.00) | 0 |
| Multi-test | 8 | 8 | 1.00 (1.00 to 1.00) | 8 | 1.00 (1.00 to 1.00) | 0 |
| Group (entity) | 10 | 3 | 0.30 (0.00 to 0.62) | 10 | 1.00 (1.00 to 1.00) | 0 |
| Temporal | 25 | 25 | 1.00 (1.00 to 1.00) | 25 | 1.00 (1.00 to 1.00) | 0 |
| Target-derived | 6 | 6 | 1.00 (1.00 to 1.00) | 6 | 1.00 (1.00 to 1.00) | 0 |
| **All types** | **89** | **80** | **0.90** | **89** | **1.00** | **0** |

*DEV numbers are optimistic by design: the rules were tuned on these files. With no false positive, precision is 1.00 for every type on DEV; that will not hold on real code.*

- **Why static-only group recall is low.** In 7 of the 10 group leaks the entity column is in the data file but never named in the code. The static pass finds the 3 where the entity comes from file paths in the code. The instrumented run finds the other 7. This is the measured value of linking the two passes.
- **Temporal leakage** is found for all three benchmark categories (common 7 of 7, differentiating 6 of 6, hard 12 of 12), including the default shuffle of train_test_split, unshuffled KFold and a time column whose name gives no

hint. The temporal baseline (scicode-lint) has not been run, so this is not yet a comparison.

- **Speed.** Static analysis took a median of 1.7 ms per file (laptop, Apple M5). An instrumented run costs one execution of the script (median 3.4 s here).
- **Real notebooks (unlabelled, DEV half only).** The static analyser ran on all 71 real Kaggle notebooks without a crash (7 had cells it had to skip). It reported medium-or-higher findings in 19 notebooks for preprocessing, 10 for multi-test, 3 each for overlap, group and temporal. These counts are descriptive only: nobody has labelled the notebooks, so no precision or recall can be stated.

## 3.6 Impact estimation: coverage and accuracy

For an injected file the true inflation is known: the metric of the leaky file minus the metric of its hand-written correct version, on the same data. LeakLens's estimate is the metric of the original run minus the metric when the run is repeated with the finding's fix. The instrumented run printed exactly the same metric as the plain run in all 96 files, so the instrumentation does not disturb the pipeline.

**Coverage: for how many correctly found leaks an estimate could be produced**

| Leakage type | Estimated | No estimate | Why no estimate |
|---|---|---|---|
| Preprocessing | 7 | 23 | pandas statistics, or data that is cross-validated |
| Overlap / duplicates | 8 | 2 | resampling before the split has no automatic fix |
| Multi-test | 0 | 8 | nested validation is not automated |
| Group (entity) | 10 | 0 | |
| Temporal | 10 | 15 | forward-looking features and whole-series statistics have no automatic fix |
| Target-derived | 6 | 0 | |
| **All types** | **41** | **48** | |

**Accuracy of the estimates (files with a single leak, DEV, data seed 0)**

| Leakage type | Metric | Findings | Mean true inflation | Mean estimate | Mean absolute error | Direction right |
|---|---|---|---|---|---|---|
| Preprocessing | test_ap | 4 | +0.0084 | +0.0000 | 0.0084 | 0% |
| Preprocessing | test_auc | 3 | +0.0000 | +0.0000 | 0.0000 | n/a |
| Overlap / duplicates | cv_auc | 1 | +0.0787 | +0.0787 | 0.0000 | 100% |
| Overlap / duplicates | test_ap | 3 | +0.3165 | +0.3167 | 0.0002 | 100% |
| Overlap / duplicates | test_auc | 2 | +0.3528 | +0.3528 | 0.0000 | 100% |
| Overlap / duplicates | test_r2 | 2 | +0.0349 | +0.0349 | 0.0000 | 100% |
| Group (entity) | cv_auc | 6 | +0.0762 | +0.0762 | 0.0000 | 100% |
| Group (entity) | test_auc | 3 | +0.0936 | +0.3933 | 0.2997 | 100% |
| Temporal | cv_mae | 3 | +0.0127 | +0.0127 | 0.0000 | 100% |
| Temporal | test_r2 | 6 | +0.0032 | +0.0032 | 0.0000 | 100% |
| Target-derived | cv_auc | 3 | +0.3703 | +0.3703 | 0.0000 | 100% |
| Target-derived | test_ap | 1 | +0.5079 | +0.5079 | 0.0000 | 100% |
| Target-derived | test_auc | 1 | +0.3475 | +0.3475 | 0.0000 | 100% |

Over the 35 estimates on bounded metrics (AUC, average precision, R2) the mean absolute error is 0.027 and the rank correlation with the true inflation is 0.96. Three things must be said about these numbers:

- **Most errors are zero because the fix is the same operation as the benchmark's own correct version**, run on the same seed. That shows the fixing machinery is right; it is not a hard test of estimation.
- The 0% in the first row is a single file: its true inflation is 0.03 in average precision and the estimate was 0, because the benchmark's correct version fits the selector after oversampling and ours before it.
- **Hold-out group splits are the weak case** (test_auc row for group: error about 0.30). With only 12 sites, the score depends strongly on which sites are held out, and LeakLens's group split holds out different sites from the reference. The direction and ranking are right; the size is not reliable. Averaging over several group splits is the planned remedy.

- **Coverage is partial:** 41 of 89 found leaks got an estimate. The rest are reported as "impact unknown" with the reason, not dropped. LeakHound's proxy estimate, the baseline for this task, has not been run yet.
- Injected inflation is built into our synthetic data. It is used only to check the estimates; it is not evidence of how severe any leakage type is in practice.

## 3.7 What went wrong, and what is missing

- **CalibraEEG case study: LeakLens found nothing.** We ran the static analyser on the companion project's 78 Python files. Expected: leakage flagged in regime A (random windows) and B (within-patient), nothing in C. Result: 0 findings in all three. That is correct for C and a miss for A and B. The reason is that this code splits with hand-written NumPy masks over custom recording objects, not with scikit-learn splitters on data frames, which is all the analyser models today. The motivating example of Chapter 1 is therefore not yet detected in its own code base.
- **The name is taken.** The novelty search found an unrelated package called `leaklens` on PyPI (first release June 2026, by another author): data-level checks plus a simple syntax check for preprocessing before a split. It does not detect group leakage from code and does not estimate impact, so the contributions stand, but the project needs a decision on its name before any release or paper.
- **A related idea exists for impact.** Leakly (2026) re-runs a user's pipeline on shuffled labels to test for leakage. It does not locate a leak or apply a fix, but our claim must be worded as per-finding impact, and Leakly must be cited.
- **The reference baseline could not be run today.** Yang et al.'s analyser built and ran on Kaggle in September; on 12 November its Datalog engine no longer installs on Kaggle's current image, so it analysed 0 of the 96 DEV files. A second attempt that builds the engine from source was started. Until it succeeds there is no measured comparison with any other tool.
- **Evaluation still to do:** TEST (needs the protocol approved), human labels for real notebooks and their agreement, baselines, the mining study, packaging. One test-split detail was seen by accident (column names of the TEST pipelines); it is logged in EVAL_CHANGES.md.
- **The benchmark and the detector have one author.** TEST uses other pipelines but the same kinds of injected leak, so even TEST will not show how the tool copes with leaks written by other people. Only the human-labelled real notebooks can show that.

## 3.8 Comparison with existing systems

The table extends Table 2.2 with the tools found since Chapter 2 and replaces "proposed" by what is implemented. It compares **capabilities**, read from each tool's paper or documentation. It does not compare accuracy: no other tool has been scored on our benchmark, and numbers that papers report on their own data (for example precision 97.6% and recall 67.8% for [3]) cannot be set against ours.

| System | Reads | Types detected | Group leakage | Temporal leakage | Impact estimate |
|---|---|---|---|---|---|
| Yang et al. [3]; LeakageDetector [6][13] | code | preprocessing, overlap, multi-test | no | no | no |
| LeakCheck, Draoui et al. [7] | code + LLM | preprocessing, overlap | not reported | not reported | no |
| scicode-lint [15] | code + LLM | 66 patterns incl. two temporal | no | shuffled CV, forward shifts | no |
| splitguard [16] | run | overlap, fit on held-out rows | only if the user declares group ids | no | no |
| Deepchecks [8], leakcheck-ml, LeakHound, dataleaks, leaklens (PyPI) [17] | data tables the user exports | duplicates, date overlap, some group checks | some, column usually named by user; no code location | date overlap on given splits | LeakHound: its own proxy model |
| Leakly [18] | re-run on shuffled labels | any (not located, not typed) | no | no | indirect, whole pipeline |

| System | Reads | Types detected | Group leakage | Temporal leakage | Impact estimate |
|---|---|---|---|---|---|
| **LeakLens (this work, implemented)** | **code, then a run** | **all six, with line and suggested fix** | **yes: from file paths or names in code; from the data at run time with nothing declared** | **yes: split and feature level, by data flow; confirmed on the data** | **yes, per finding, by re-running the user's script with the fix (6 kinds of fix)** |

*"no" and "not reported" mean the source we read does not state support. Tools known only from PyPI were read from package metadata and README (literature/related_work.csv).*

## 3.9 Objectives of Chapter 1

| # | Objective | Status on 12 November |
|---|---|---|
| O1 | Taxonomy of six types with a rule per type | Met (docs/TAXONOMY.md), with runnable examples that are part of the test suite. |
| O2 | Hybrid static + dynamic analyser with line, type and fix | Built and working on DEV. Not yet evaluated on unseen code. |
| O3 | Impact of each finding by fixing and re-running | Built for six kinds of fix; an estimate for 41 of 89 DEV leaks. |
| O4 | Labelled benchmark; precision of at least 0.90 and recall above 67.8% on shared types | Benchmark built. On DEV the targets are exceeded, but DEV is the tuning split: **the target is not yet demonstrated**. It must be shown on TEST and on labelled real notebooks. |
| O5 | Empirical study on public notebooks | Not started. 150 notebooks sampled; none labelled. |
| O6 | Usable software | CLI, JSON, SARIF and exit codes done. No PyPI release, pre-commit hook or GitHub Action yet. |

## 3.10 Next steps and decisions needed

- **Approve (or change) the evaluation protocol** (docs/EVAL_PROTOCOL_SUMMARY.md, one page). TEST is scored once, straight after approval.
- **Label real notebooks:** two people, in the prepared random order, without seeing the tool's output.
- **Decide the project name** in the light of the existing PyPI package.
- Get at least one baseline running on the benchmark (Yang et al. from source; data-level tools on the splits the instrumented run materialises; LeakHound for impact).
- Extend the analyser to hand-written splits (NumPy masks, custom loaders) so that the CalibraEEG case is found; average group-split impact over several splits; add fixes for the leak kinds that have none.

### References


[1] S. Kaufman et al. Leakage in data mining: formulation, detection, and avoidance. ACM TKDD 6(4), 2012. https://doi.org/10.1145/2382577.2382579

[2] S. Kapoor, A. Narayanan. Leakage and the reproducibility crisis in machine-learning-based science. Patterns 4(9):100804, 2023. https://doi.org/10.1016/j.patter.2023.100804

[3] C. Yang, R. A. Brower-Sinning, G. Lewis, C. Kaestner. Data leakage in notebooks: static detection and better processes. ASE 2022. https://doi.org/10.1145/3551349.3556918

[5] S. Roth. Which leakage types matter? A quantitative landscape across 2,047 benchmark datasets. arXiv:2604.04199, 2026.

[6] E. A. AlOmar et al. LeakageDetector: an open source data leakage analysis tool in machine learning pipelines. 2025. https://arxiv.org/abs/2503.14723

[7] T. Draoui, M. W. Mkaouer, C. Newman. Improving data leakage detection in machine learning notebooks through static slicing and structured LLM prompts. ACM, 2026. https://doi.org/10.1145/3808199

[8] S. Chorev et al. Deepchecks: a library for testing and validating machine learning models and data. JMLR 23(285), 2022.

[13] LeakageDetector 2.0: analyzing data leakage in Jupyter-driven machine learning pipelines. 2025. https://arxiv.org/abs/2509.15971

[15] Samsonau. scicode-lint: detecting methodology bugs in scientific Python code with LLM-generated patterns. arXiv:2603.17893, 2026.

[16] T. Aiello. splitguard 0.1.0. PyPI, 2026. https://github.com/Tommasoaiello13/splitguard

[17] Prajwal. leaklens 1.3.0 (unrelated package of the same name). PyPI, 2026. https://pypi.org/project/leaklens/

[18] DeMONLab-BioFINDER. Leakly 0.1.2. PyPI, 2026. https://github.com/DeMONLab-BioFINDER/Leakly


[1] to [14] as in Chapters 1 and 2. [16] to [18]: package metadata and README only. Result numbers are read from results/2026-11-12_dev_injected_leaklens_v2.